DAY 5 | HANDS-ON ACTIVITIES

API Fundamentals and FastAPI Core: Problem Statements

Eight problems. Use TestClient in Colab, or run uvicorn and use /docs in VS Code.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Design the Endpoints

Level: Warm-up | **Concepts:** REST design, methods, status codes

For a library app with books and members, list 8 endpoints with method, URL, purpose and success status code. No code needed.

Sample run / expected result

```
GET /books           list books      200
POST /books          add a book      201
DELETE /books/{id}   remove a book   204
```

Hints

- Use plural nouns and no verbs in URLs.

Problem 2: Hello API

Level: Easy | **Concepts:** FastAPI setup, GET

Create a FastAPI app with GET / returning a welcome message and GET /health returning {"status": "ok"}. Run it and open /docs.

Sample run / expected result

```
GET /health -> 200 {"status": "ok"}
```

Hints

- uvicorn main:app --reload
- In Colab: TestClient(app).get("/health").json()

Problem 3: Parameters Playground

Level: Easy | **Concepts:** path, query, Query()

Add GET /square/{n} returning n and n squared, and GET /books with an optional author and a limit between 1 and 50 (default 10). Try invalid values and note the status codes.

Sample run / expected result

```
GET /square/7 -> {"n": 7, "square": 49}
GET /square/abc -> 422
GET /books?limit=500 -> 422
```

Hints

- limit: int = Query(10, ge=1, le=50)

Problem 4: Quotes API

Level: Medium | **Concepts:** GET list, GET by id, 404, POST 201, DELETE 204

Store 5 quotes in a dict. Add GET /quotes, GET /quotes/{id} (404 if missing), GET /quotes/random, POST /quotes (201) and DELETE /quotes/{id} (204).

Sample run / expected result

```
GET /quotes/99 -> 404 {"detail": "Quote not found"}
DELETE /quotes/6 -> 204
```

Hints

- Put /quotes/random above /quotes/{qid} so it matches first.

Problem 5: Book Models

Level: Easy | **Concepts:** Pydantic BaseModel, Field

Define BookCreate (title min 1 char, author, pages > 0, year between 1450 and 2026), BookUpdate (all optional) and Book (adds id). Create valid and invalid objects and print the errors.

Sample run / expected result

```
BookCreate(title="", ...) -> ValidationError: title too short
```

Hints

- from pydantic import ValidationError

Problem 6: Books CRUD

Level: Challenging | **Concepts:** POST body, response_model, PATCH, 404, APIRouter

Build full CRUD for books in a dict, on an APIRouter with prefix /books and tag Books. PATCH must change only the fields sent.

Sample run / expected result

```
PATCH /books/1 {"pages": 250} -> 200 with only pages changed
GET /books/99 -> 404
```

Hints

- data.model_dump(exclude_unset=True)

Problem 7: Review a Codex Endpoint NEW (v4)

Level: Medium | **Concepts:** AI code review, tests

Ask OpenAI Codex for a DELETE /books/{id} endpoint using a precise prompt (method, path, status codes, errors, tests). Review the diff with the checklist from class. Write down one thing you changed or confirmed, and add a test that fails if the endpoint returns 200 instead of 204.

Sample run / expected result

```
Review note: Codex returned 200 with a body; changed to status_code=204.
test_delete_returns_204 ... passed
test_delete_unknown_404 ... passed
```

Hints

- Ask for one endpoint at a time.
- Run the test against the unreviewed code first. Does it fail?

Problem 8: Test Everything

Level: Stretch | **Concepts:** TestClient, assertions

Write at least 10 assert-based tests with TestClient that check status codes and bodies for all your endpoints, including error cases.

Sample run / expected result

```
assert client.get("/books/99").status_code == 404
All 10 tests passed
```

Hints

- from fastapi.testclient import TestClient